Project: Build Your Own MD Simulation

Course: Electronic Structure & Atomistic Modeling of Materials

April 5, 2026

Molecular Dynamics (MD) is a computer-based simulation method that provides a window into the atomic world. By calculating the forces acting on every atom in a system and solving Newton's equations of motion, we can determine the trajectories of these atoms over time. From these trajectories, we can extract macroscopic properties like temperature, pressure, and structural patterns. In this project, you will build a functional MD "engine" from scratch using Python. You will learn how to:

1. Define atomic interactions.
2. Implement periodic boundary conditions.
3. Implement numerical integration.
4. Optimize the code for performance.
5. Use statistical mechanics to drive a system through a phase transition.

By the end, you should be able to simulate a block of Solid inert gas and observe it melting into a Liquid state.

While the Molecular Dynamics engine is developed using general reduced units, each group will evaluate their results for one inert solid system and one property to answer the final Module:

- Pick any one species: Neon (Ne), Argon (Ar), Krypton (Kr), or Xenon (Xe)
- Pick any one property in Module 6: Radial distribution function or Mean square displacement.

To map your dimensionless simulation data back to real-world physical properties in Module 6, refer to Table 5.1 for the necessary ϵ and σ parameters, and for lattice parameters, you can refer to Table 5.2 in LeSar's book chapter 5.

Project Submission Guidelines

Each group must submit a final report as a PDF of your results/plots/tables (as required) and the explanation of your results, along with the Jupyter Notebook containing the codes for:

1. **Module-wise Implementation and Testing:** Submit the source code for each of the six modules and each module must be accompanied by its specific test results.
2. **Integrated Molecular Dynamics Engine:** Submit the final, assembled MD main loop code. You must include diagnostic plots to demonstrate the energy conservation and numerical stability of your MD simulations.
3. **Physical Property Analysis (Chosen Inert Gas):** Submit the final results for your assigned system (Ne, Ar, Kr, or Xe) using the parameters from **Table 5.1**. This must include:
   - **Radial Distribution Function:** A plot of $g(r)$ vs. r (scaled to Å). Briefly describe how this plot confirms whether your system is in a solid or liquid state.

- **Mean Square Displacement (MSD):** A plot of the MSD vs. time (scaled to picoseconds) using *unwrapped* coordinates. Calculate the diffusion coefficient (D) from the slope if the system is in the liquid phase.

Important: Grading Weight & AI Usage Policy

To ensure this project evaluates your understanding of computational physics rather than just programming syntax, the final project marks are distributed as follows:

1. **Code Implementation (25% of marks):** The functionality and accuracy of your Python modules.
2. **Physics Analysis & Presentation (75% of marks):** The quality of your analysis, plots, physical justification, and the final presentation of your results.

AI Co-Pilot Rule:
You may use AI tools to assist you with coding syntax and debugging. However, if you use AI, you **must** include an **AI Acknowledgement Appendix** in your final report. This appendix must explicitly highlight:

a. The exact prompts you used to query the AI.
b. The raw code that the AI generated.
c. A short paragraph explaining how you had to modify the AI's code to make it physically correct for this specific assignment.

Module 1: The Physics of Interaction

Learning Objectives

- Understand how atomic motion is driven by potential energy.
- Implement the Lennard-Jones (LJ) potential as a model for an inert system.
- Master the concept of Reduced (Dimensionless) Units.

2. Basics of Molecular Dynamics

The core of MD is solving Newton's Second Law for every atom i in a system of N particles:

$$m_i \frac{d^2 r_i}{dt^2} = F_i \tag{1}$$

In MD, the force F_i is a conservative force, meaning it is derived from the gradient of the potential energy U with respect to the atom's position:

$$F_i = -\nabla_i U(r^N) \tag{2}$$

3. The Pair Potential Approximation

Calculating the total energy of a material is complex, but for many systems, we can approximate it by summing the interactions between every unique pair of atoms. For an inert system, we use the Lennard-Jones (LJ) Potential, which models the attraction (at long range) and repulsion (at short range) between closed-shell atoms. Lennard-Jones (LJ) Potential (Standard Units):

$$\phi(r_{ij}) = 4\epsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right] \tag{3}$$

- σ: The distance where the potential is zero.
- ϵ: The depth of the potential well (strength of interaction).

4. Why use Reduced Units?

Standard units (SI or CGS) involve extremely small numbers (masses of 10^{-27} kg, distances of 10^{-10} m) that can lead to numerical errors and code that is difficult to debug. Instead, we use Reduced Units where we scale all values by the fundamental parameters σ, ϵ, m. In reduced units, the LJ potential becomes elegantly simple:

$$\phi^*(r^*) = 4 \left[\left(\frac{1}{r^*} \right)^{12} - \left(\frac{1}{r^*} \right)^{6} \right] \tag{4}$$

5. Reference reading sections:

- LeSar: Section 5.3.1 (LJ model); Section 6.1 (MD Basics); Section 6.2 (Reduced Units and Table 6.1)
- Allen & Tildesley: Section 1.3.2 (Atomic systems)

6. Coding Task: Module 1

Your goal is to write a function that calculates the Potential Energy and the Force between two atoms. Requirements:

1. Use Reduced Units ($r^* = r/\sigma$).
2. Implement a Shifted Potential: To avoid a jump in energy at the cutoff distance (r_c), you

must subtract the value of the potential at the cutoff.

```
# Module 1 Coding Task: Interatomic Interactions
import numpy as np

def calculate_lj_properties(r_sq, r_cutoff_sq):
    """
    Calculates reduced potential energy and force magnitude for a pair
        of atoms.
    TASK: Use the equations to implement the Lennard-Jones potential and
         force.
    REQUIREMENTS:
    1. Implement the shifted potential: phi(r) - phi(rc).
    2. Return the force magnitude divided by r (force/r).
    """
    # YOUR CODE HERE
```

Module 2: Boundary Conditions and Geometry

Learning Objectives

- Understand how to simulate a "bulk" material using a finite number of atoms.
- Implement Periodic Boundary Conditions (PBC) to eliminate surface effects.
- Apply the Minimum Image Convention to ensure consistent interatomic interactions.
- Generate a stable starting geometry using a Face-Centered Cubic (FCC) lattice.

2. Periodic Boundary Conditions (PBC)

In a real material, atoms in the "bulk" are surrounded by other atoms in every direction. However, simulations are limited to a small number of particles (e.g., $N = 864$). If we simply placed these in a box, atoms at the edges would behave differently than those in the center because they have no neighbors on one side. To solve this, we use Periodic Boundary Conditions. We treat our simulation cell as if it were surrounded by identical replicas of itself in all directions. When an atom leaves the box through one side, it immediately re-enters from the opposite side with the same velocity. Periodic Boundary Wrapping: When an atom's coordinate (x) moves outside the primary simulation cell [0, L], it must be wrapped back into the box:

```
if x > L: x = x - L
if x < 0: x = x + L
```

3. The Minimum Image Convention

In this infinite repeating universe, an atom i could technically interact with infinitely many copies of atom j. To keep the physics local and computationally feasible, we adopt the Minimum Image Convention: an atom interacts only with the nearest image of every other atom in the system. Key Equation: The shortest distance dx between two atoms in a box of length L is calculated as:

$$dx_{ij} = x_i - x_j - L \cdot \text{round}\left(\frac{x_i - x_j}{L}\right) \tag{5}$$

4. Geometry: The FCC Lattice

Starting atoms in random positions often leads to "unphysical overlaps" where two atoms are so close that their repulsive force is near-infinite, causing the simulation to crash. To avoid this, we start atoms

on a regular Face-Centered Cubic lattice. An FCC unit cell contains 4 atoms at these relative positions:

$$
\begin{aligned}
b_1 &= (0, 0, 0) \\
b_2 &= (0.5, 0.5, 0) \\
b_3 &= (0.5, 0, 0.5) \\
b_4 &= (0, 0.5, 0.5)
\end{aligned}
$$

By repeating this unit cell n times in the x, y, z directions, we create a simulation cell with $N = 4n^3$ atoms.

5. Reference reading sections:

- LeSar: Section 3.4 (PBC); Section 3.5 (Implementation); and Appendix B.2 (fcc structures).
- Allen & Tildesley: Section 1.5.2 (PBC Theory) and Section 1.5.4 (Coding Minimum Image)

6. Coding Task: Module 2

Your goal is to write two functions: one to initialize your atoms on a lattice and one to handle the periodic geometry. Requirements:

- Lattice Generator: Create a function that places $N = 4n^3$ atoms on an FCC lattice given a target density ρ^*.
- Distance Logic: Create a function that calculates the distance between two atoms while accounting for the Minimum Image Convention.

```
# -- Module 2 Coding Task: Geometry and Boundaries
import numpy as np

def generate_fcc_lattice(n_cells, rho_star):
    """
    Generates an FCC lattice of atoms.
    TASK:
    1. Calculate the box length L based on density (rho = N/V).
    2. Place 4 atoms in each of the n_cells^3 unit cells.
    """
    # YOUR CODE HERE

def apply_minimum_image(pos_i, pos_j, L):
    """
    Calculates the shortest distance vector between two atoms.
    TASK: Use the rounding logic to ensure dx,
dy, and dz
    are between -L/2 and L/2.
    """
    # YOUR CODE HERE
```

Module 3: The Velocity Verlet Integrator

Learning Objectives

- Understand numerical integration as a "time machine" for atomic systems.
- Implement the Velocity Verlet algorithm to solve Newton's equations of motion.
- Learn to diagnose code accuracy through Total Energy conservation.

2. The Numerical Integration Challenge

The movement of atoms is governed by Newton's second law ($F = ma$). Because the forces between atoms change constantly as they move, there is no analytical solution to predict their positions at any future time. Instead, we use finite difference methods. We break time into very small, discrete steps (δt). At each step, we calculate the current forces, move the atoms a tiny bit, and repeat. The challenge is to use an algorithm that is self-consistent—one where the accelerations calculated from positions match the accelerations used to move them.

3. The Velocity Verlet Algorithm

While there are many integration schemes, the Velocity Verlet algorithm is the standard in MD because it is numerically stable, time-reversible, and synchronizes positions and velocities at the same time step. The algorithm proceeds in a loop:

1. **Update Positions:** Move atoms based on current velocity and acceleration.

$$r_i(t + \delta t) = r_i(t) + v_i(t)\delta t + \frac{1}{2}a_i(t)\delta t^2 \tag{6}$$

2. **Half-Step Velocity:** Update velocity to the midpoint of the time step.

$$v_i(t + \frac{1}{2}\delta t) = v_i(t) + \frac{1}{2}a_i(t)\delta t \tag{7}$$

3. **Update Forces:** Calculate new forces based on the new positions (using your code from Module 1).
4. **Full-Step Velocity:** Complete the velocity update using the new forces.

$$v_i(t + \delta t) = v_i(t + \frac{1}{2}\delta t) + \frac{1}{2}a_i(t + \delta t)\delta t \tag{8}$$

4. Reliability Check: Energy Conservation

In a standard MD simulation (the NVE ensemble), the total energy ($E = K + U$) must remain constant.

- **Kinetic Energy (K):** Energy of motion, derived from velocities.

$$K = \sum_{i=1}^{N} \frac{1}{2} m_i v_i^2 \tag{9}$$

- **Potential Energy (U):** Energy of position, derived from interatomic distances.

If your integrator is working correctly, energy will oscillate slightly but show no systematic drift over time. A "drifting" energy usually indicates a time step (δt) that is too large or a bug in the logic.

5. Additional Module 3 Nuance: The Reduced Time Scale

In this module, it is critical to explicitly define the Reduced Time unit (t^*) to ensure numerical stability.

- Reduced Time Definition: $t^* = t/t_0$, where the reference time is $t_0 = \sigma\sqrt{m/\epsilon}$.
- Choosing t^*: For the Velocity Verlet algorithm to be stable, you typically need approximately 50 integration points per vibrational period.
- Stability Metric: A "safe" time step is one where the fluctuations in total energy (E) are roughly 1 part in 10^4 of the average energy. If your energy shows a systematic "climb" or "drift," your δt^* is likely too large.

6. Reference reading sections:

- LeSar: Section 6.1.1 (Numerical integration), Section 6.1.2 (Conservation laws), and Section 6.1.3 (Reliability).
- Allen & Tildesley: Section 3.2.1 (The Verlet algorithm).

7. Coding Task: Module 3

Your goal is to implement the Velocity Verlet update step. In reduced units, we assume mass $m = 1$, so acceleration $a = F$.

```
# Module 3 Coding Task: The Integration Engine
import numpy as np

def velocity_verlet_step(pos, vel, force, dt, box_length):
    """
    Advances the system by one time step dt using Velocity Verlet.
    TASK:
    1. Update positions.
    2. Apply Periodic Boundary Conditions (wrap atoms back into box).
    3. Update velocities (half-step).
    4. Calculate new forces (This will require your Module 1 & 2 logic).
    5. Update velocities (full-step).
    """
    # YOUR CODE HERE

 # Step 1: New Positions
    # pos_new = pos + vel*dt + 0.5*force*dt**2
    # Step 2: PBC Wrap (Module 2 logic)
    # pos_new = pos_new % box_length
    # Step 3: Half-step velocity
    # vel_mid = vel + 0.5*force*dt
    # Step 4: New Forces (Call your Module 1 force function here)
    # force_new = calculate_total_force(pos_new, box_length)
    # Step 5: Full-step velocity
    # vel_new = vel_mid + 0.5*force_new*dt
```

Module 4: Performance and Optimization

Learning Objectives

- Identify the computational bottleneck in MD simulations.
- Implement Verlet Neighbor Lists to reduce the scaling of force calculations from $O(N^2)$ to $O(N)$.
- Apply NumPy Vectorization to accelerate mathematical operations in Python.

2. The $O(N^2)$ Bottleneck

In the previous modules, your force calculation likely used nested loops to check every pair of atoms i and j. This means for N atoms, you perform roughly $N^2/2$ distance checks. As you increase the number of atoms to 864 or more, the simulation will slow down exponentially. Since the interatomic potential is truncated at a cutoff distance ($r_c = 2.5$), atoms further apart than r_c contribute zero force. Calculating their distances every single time step is a waste of CPU cycles.

3. Verlet Neighbor Lists

A Verlet List stores a list of all atoms j within a "skin" radius (r_s) around atom i, where r_s is slightly larger than the cutoff r_c:

$$r_s = r_c + \text{skin} \tag{10}$$

The Trick: You only calculate forces for pairs in this list. The Update: You only need to rebuild the list when an atom has moved far enough that a new neighbor might have entered the cutoff zone. This happens when the maximum displacement of any particle exceeds half the skin thickness:

$$\Delta r_{max} > \frac{r_s - r_c}{2} \tag{11}$$

4. Vectorization with NumPy

Python is slow with "for-loops." To make your code efficient, you must use vectorization—performing operations on entire arrays at once. For example, instead of looping through x, y, z to find a distance, you can use NumPy to subtract two coordinate arrays and find the magnitude in a single operation.

5. Additional Module 4 Nuance: Force Continuity

While Module 1 introduced the Shifted Potential, optimizing for performance requires looking at the Force Continuity.

- Discontinuity Issues: Even with a shifted potential, a "jump" in the force magnitude often exists at the cutoff distance (r_c).
- Shifted-Force Potential: This discontinuity can act as a source or sink of energy as atoms cross the cutoff boundary, leading to poor energy conservation. To achieve accuracy, one can implement a Shifted-Force potential that ensures both the energy and the force go smoothly to zero at r_c.

6. Reference reading sections:

- LeSar: Section 3.5 (Implementation of neighbor lists).
- Allen & Tildesley: Section 5.2.2 (Avoiding the square root) and Section 5.3 (Neighbour lists).

7. Coding Task: Module 4

Your goal is to refactor your force routine and implement a neighbor list system. Requirements:

1. Vectorized Forces: Rewrite your force function to use NumPy array operations instead of nested Python loops.
2. Neighbor List: Write a function `update_neighbor_list` that finds all pairs within r_s and stores them.
3. Efficiency Check: Only call the update function when the displacement threshold is reached.

```python
# Module 4 Coding Task: Performance Optimization
import numpy as np

def update_neighbor_list(pos, L, r_skin_sq):
    """
    Builds a list of atom pairs within the skin radius.
    TASK: Loop through all atoms and store indices (i, j) if
    their distance is less than r_skin.
    Use the Minimum Image Convention!
    """
    # YOUR CODE HERE

def calculate_forces_optimized(pos, neighbor_pairs, L, r_cutoff_sq):
    """
    Calculates total forces using ONLY the pairs in the neighbor list.
```

```
    TASK: Implement the
LJ force logic from Module 1, but iterate
    only over neighbor_pairs.
    """
    # YOUR CODE HERE
```

Module 5: Initialization, Equilibration, and System Assembly

Learning Objectives

- Establish a physically valid starting state for the system.
- Apply Statistical Mechanics to initialize velocities from a Maxwell-Boltzmann distribution.
- Implement Velocity Rescaling to drive the system toward a target temperature.
- Assemble the "Main Loop" of an MD simulation following the master algorithm.

2. Starting Up: Positions and Velocities

A simulation cannot simply "start." It needs a valid initial state.

- Positions: As implemented in Module 2, we start atoms on an fcc lattice to avoid unphysical overlaps that would cause infinite forces.
- Velocities: Atoms in a real material have a range of speeds described by the Maxwell-Boltzmann distribution. We assign each velocity component (v_x, v_y, v_z) from a Normal (Gaussian) distribution with a standard deviation related to the target temperature.

Velocity Distribution (Maxwell-Boltzmann): Each component $v_{i\alpha}$ is chosen from:

$$\rho(v_{i\alpha}) = \frac{1}{\sigma\sqrt{2\pi}} e^{-v_{i\alpha}^2/2\sigma^2} \tag{12}$$

In reduced units, $\sigma = \sqrt{T^*}$.

- Zeroing Momentum: To prevent your entire block of the system from drifting out of the simulation box, you must shift all velocities so the total linear momentum is zero.

3. Reaching Equilibrium (Equilibration)

When you start a simulation, the system is in an artificial state (a perfect lattice). As it begins to move, the potential and kinetic energies will fluctuate wildly as the atoms "relax". To reach a specific target temperature (T^*):

$$T^* = \frac{\sum_{i=1}^{N} v_i^2}{3N} \tag{13}$$

we use Velocity Rescaling. If the current instantaneous temperature is too low, we multiply all velocities by a factor to speed them up until the target is reached:

$$v_{\text{new}} = v_{\text{old}} \sqrt{\frac{T^*_{\text{target}}}{T^*_{\text{inst}}}} \tag{14}$$

4. The Master MD Algorithm

A standard MD code follows these steps:

1. Initialization: Set positions and velocities.

2. Equilibration Phase: Run the integrator and periodically rescale velocities to reach the target T^*.
3. Production Phase: Turn off rescaling and begin accumulating data for analysis.

5. Additional Module 5 Nuance: Energy Compensation

During the equilibration phase, you should carefully monitor the exchange between different energy types.

- Potential vs. Kinetic: As the system relaxes from the artificial fcc lattice, potential energy (U^*) and kinetic energy (K^*) can vary significantly.
- Exact Compensation: In a physically correct simulation, these variations must exactly compensate for one another to keep the total energy E^* a flat line. You should plot all three values on one graph; any fluctuation in total energy that mirrors U^* or K^* suggests a bug in the integration or force calculation.

6. Reference reading sections:

- LeSar: Section 6.1.5 (Initial conditions), Section 6.1.6 (Steps in an MD simulation), and Section 6.3 (Velocity rescaling).
- Allen & Tildesley: Section 5.7.1 (Initial configuration) and Section 5.7.2 (Initial velocities).

7. Coding Task: Module 5

Your goal is to write the initialization routines and the master loop that assembles your code from Modules 1–4. Requirements:

1. Velocity Initializer: Create a function that assigns random Gaussian velocities and then shifts them to ensure the total system momentum is zero.
2. Thermostat: Create a simple velocity rescaling function.
3. The Assembly: Write a loop that runs the simulation for 2000 steps, applying rescaling every 10 steps for the first 1000 steps (Equilibration), then running freely for the remaining 1000 steps (Production).

```python
# Module 5 CodingTask: System Assembly
import numpy as np

def initialize_velocities(N, T_target):
    """
    Assigns initial velocities from a Maxwell-Boltzmann distribution.
    TASK:
    1. Generate Gaussian random numbers for vx, vy, vz with std_dev =
       sqrt(T_target).
    2. Calculate total momentum and subtract the average from each
       particle
       so the center-of-mass is stationary.
    """
    # --- YOUR CODE HERE ---

def rescale_velocities(vel, T_target):
    """
    Scales velocities to match the target temperature.

  """
    # 1. Calculate current T_inst = (sum v^2) / 3N
    # 2. Scale factor lambda = sqrt(T_target / T_inst)
    # 3. vel = vel * lambda
    # --- YOUR CODE HERE ---
```

```
# --- THE MAIN MD LOOP ---
# 1. Setup: Define N, density, T_target, dt, and steps.
# 2. Initialize: Call generate_fcc_lattice and initialize_velocities.
# 3. Main Loop:
#    for step in range(total_steps):
#        - Call velocity_verlet_step (from Module 3)
#        - If in Equilibration and step % 10 == 0:
#            - Call rescale_velocities
#        - Accumulate Energy and Temperature data for plotting.
```

Module 6: Final Analysis — Melting and Diffusion

Learning Objectives

- Structural Analysis: Use the Radial Distribution Function $g(r)$.
- Dynamical Analysis: Calculate the Mean Square Displacement (MSD) to determine the Self Diffusion Coefficient (D).

2. Identifying the State: $g(r)$

The thermodynamics (Temperature and Pressure) of a system do not always tell you if it is a solid or a liquid. We need structural metrics:

- Radial Distribution Function $g(r)$: This represents the probability of finding an atom at a distance r from a central atom.

$$g(r) = \frac{V}{4\pi r^2 N^2} \left\langle \sum_i \sum_{j \neq i} \delta(r - r_{ij}) \right\rangle \tag{15}$$

- Solid: Shows sharp, distinct peaks corresponding to shells of neighbors in a lattice.
- Liquid: Shows broad, decaying peaks that eventually asymptote to 1.0, indicating no long-range order.

3. Measuring Motion: MSD and Diffusion

Molecular Dynamics is uniquely suited to study transport properties because it tracks the actual trajectories of atoms.

- Mean Square Displacement (MSD): This measures how far atoms have wandered from their original positions over time.

$$\langle R^2(t) \rangle = \frac{1}{N} \sum_{i=1}^{N} |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \tag{16}$$

- Diffusion Coefficient (D): In a liquid, the MSD grows linearly with time. The slope of this line is directly proportional to D via the Einstein relation:

$$D = \lim_{t \to \infty} \frac{1}{6t} \langle R^2(t) \rangle \tag{17}$$

Note: To calculate MSD correctly, you must use "unwrapped" coordinates (positions that do not "jump" across boundaries) to avoid artificial resets in distance. The Problem with "Wrapped" Coordinates is that in standard Periodic Boundary Conditions, when an atom leaves one side of the box, it "jumps" to the opposite side (e.g., its position coordinate might suddenly change from L to 0). Why

it breaks MSD? If you use these wrapped coordinates, the MSD calculation will see this jump as a massive, instantaneous displacement. This creates artificial "spikes" in your data that do not reflect the physical diffusion of the atom. The Solution: You must maintain a set of unwrapped coordinates. These coordinates ignore the periodic boundaries and allowed the atom's position to grow indefinitely (e.g., x can become $2L, 3L$, etc.), representing the actual path the molecule has traveled through space

4. Additional Module 6 Nuance: Statistical Reliability (Binning)

Because MD time steps are highly correlated (the state at step t is very similar to $t+1$), standard statistics can underestimate errors.

- Binning Procedure: The set of all time steps must be divided into subsets (or "bins") that are long enough so the properties at the start and end of the bin are uncorrelated.
- Reporting Results: Calculate the average for each bin and then determine the variance of the mean from those bin averages. Final thermodynamic properties should be reported with an error bar.

5. Reference reading sections:

- LeSar: Section 6.2.2 (Analysis), Section 6.9.3 (Calculating radial distribution function), and Appendix G.7 (Correlation Functions).
- Allen & Tildesley: Section 6.2 (Liquid structure) and Section 6.3 (Time correlation).

6. Coding Task: Module 6

Your goal is to write the initialization routines and the master loop that assembles your code from Modules 1–4. **Pick any ONE of the tasks, either the structural analysis or the diffusion analysis**. Requirements:

1. Structural Analysis — Radial Distribution Function $g(r)$
   - Implementation: Write a function to calculate the Radial Distribution Function, which represents the probability of finding an atom at a distance r from a central atom.
   - Distance Calculation: For each stored configuration, loop through all unique pairs of atoms and calculate their separation distance using the Minimum Image Convention to account for periodic boundaries.
   - Histogramming: Sort these distances into a histogram with a specific bin width Δr (typically 50–100 bins up to a maximum distance of half the box length)
   - Normalize the histogram by dividing each bin by the volume of the corresponding spherical shell ($4\pi r^2 \Delta r$) and the global number density to ensure the function asymptotes to 1.0 at large distances.
2. Dynamical Analysis — Mean Square Displacement (MSD).
   - Unwrapped Coordinates: You must implement a routine to track "unwrapped" coordinates, which ignore periodic boundary jumps and allow coordinates to grow indefinitely.
   - Avoiding Artifacts: Wrapped coordinates must be avoided for this calculation because they create artificial "spikes" in displacement whenever an atom crosses a boundary.
   - Calculation: Write a function to compute the Mean Square Displacement (MSD) by averaging the squared distance each atom has traveled from its initial position at $t = 0$.

```
# --- Module 6 Coding Task: Final Analysis (pick any one) ---
import numpy as np
import matplotlib.pyplot as plt

def calculate_rdf(pos, box_length, dr, r_max):
    """
```

```
    Calculates the Radial Distribution Function g(r).
    TASK:
    1. Loop through all pairs and calculate distance r using MIC.
    2. Sort distances into a histogram with bin width dr.
    3. Normalize by the volume of the spherical shells and density.
    """
    # --- YOUR CODE HERE ---

def calculate_msd(current_pos_unwrapped, initial_pos):
    """

  Calculates the Mean Square Displacement.
    TASK: Calculate the average squared distance of all particles
    from their starting positions.
    """
    # --- YOUR CODE HERE ---

# --- FINAL PROJECT CHALLENGE: Melting of an Inert system ---
# 1. Run a simulation starting from an fcc lattice at T* = 0.2.
# 2. Increase temperature in steps (0.2 -> 0.4 -> 0.6 -> 0.8 -> 1.0).
# 3. For each temperature, equilibrate and then collect g(r) data.
# 4. Plot all g(r) curves on one graph.
# 5. Extract D from your MSD vs. Time plot in the liquid state.
```

Additional Checklist for the project:

1. Does the Total Linear Momentum (P) remain at zero?
2. Is the total energy free of systematic drift?
3. Are total energy fluctuations less than 1 part in 10^4?
4. Does the Radial Distribution Function $g(r)$ asymptotically reach 1.0 at large r?
5. Are you using "unwrapped" coordinates for the Mean Square Displacement?